TI-MSP430FR6989

Estructura de computadores

Grado en Informática

Universidad Complutense de Madrid

Oscar Garnica. Versión 2021-22

MSP430FR6989

- Microcontrolador RISC de 16 bits @16MHz
- Arquitectura registro-registro
- Siete modos de direccionamiento
  - Solo cuatro modos de direccionamiento operandos destino
  - Unidad mínima direccionable: byte
- 16 registros
  - R0: PC
  - R1: SP
  - R2: SR
  - R3: generador constantes (CG)

Oscar Garnica. Versión 2021-22

MSP430FR6989

- Periféricos mapeados en memoria
  - Accesibles mediante buses de datos y direcciones
- ISA con 51 instrucciones con tres formatos distintos
- Formato datos
  - Palabra: 16 bits
  - Byte
- 10 puertos GPIO. Algunos con capacidad de generar INT
- 3 canales DMA
- RTC

Oscar Garnica. Versión 2021-22

MSP430FR6989. Registro de estado

- Registro fuente y destino de instrucciones

15 … 9	8	7							0
Reserved	V	SCG1	SCG0	OSC OFF	CPU OFF	GIE	N	Z	C

Bit	Description	
Reserved	Reserved	
V	Overflow. This bit is set when the result of an arithmetic operation overflows the signed-variable range.	
	`ADD(.B), ADDX(.B,.A), ADDC(.B), ADDCX(.B.A), ADDA`	Set when: positive + positive = negative negative + negative = positive otherwise reset
	`SUB(.B), SUBX(.B,.A), SUBC(.B),SUBCX(.B,.A), SUBA, CMP(.B), CMPX(.B,.A), CMPA`	Set when: positive – negative = negative negative – positive = positive otherwise reset
SCG1	System clock generator 1. This bit may be used to enable or disable functions in the clock system depending on the device family; for example, DCO bias enable or disable.	
SCG0	System clock generator 0. This bit may be used to enable or disable functions in the clock system depending on the device family; for example, FLL enable or disable.	
OSCOFF	Oscillator off. When this bit is set, it turns off the LFXT1 crystal oscillator when LFXT1CLK is not used for MCLK or SMCLK. In FRAM devices, CPUOFF must be 1 to disable the cyrstal oscillator.	
CPUOFF	CPU off. When this bit is set, it turns off the CPU and requests a low-power mode according to the settings of bits OSCOFF, SCG0, and SCG1.	
GIE	General interrupt enable. When this bit is set, it enables maskable interrupts. When it is reset, all maskable interrupts are disabled.	
N	Negative. This bit is set when the result of an operation is negative and cleared when the result is positive.	
Z	Zero. This bit is set when the result of an operation is 0 and cleared when the result is not 0.	
C	Carry. This bit is set when the result of an operation produced a carry and cleared when no carry occurred.	

Oscar Garnica. Versión 2021-22

Excepciones

- Excepciones **vectorizadas** no-enmascarables y enmascarables
  - RESET: excepción más prioritaria. **NO** enmascarable. Lleva al micro a un estado inicial. Debida a un error severo de funcionamiento
  - No enmascarables: interrupciones de alta prioridad. Gestionadas por registros específicos
  - Enmascarables globalmente y localmente: menor prioridad. I/O.
- Cada excepción tiene asignada un vector y una prioridad

INTERRUPT SOURCE	INTERRUPT FLAG	SYSTEM INTERRUPT	WORD ADDRESS	PRIORITY
System Reset Power up, Brownout, Supply Supervisor External Reset RST Watchdog time-out (watchdog mode) WDT, FRCTL MPU, CS, PMM password violation FRAM uncorrectable bit error detection MPU segment violation FRAM access time error Software POR, BOR	SVSHIFG PMMRSTIFG WDTIFG WDTPW, FRCTLPW, MPUPW, CSPW, PMMPW UBDIFG MPUSEGIIFG, MPUSEG1IFG, MPUSEG2IFG, MPUSEG3IFG ACCTEIFG PMMPORIFG, PMMBORIFG (SYSRSTIV) (1) (2)	Reset	0FFFEh	Highest
System NMI Vacant memory access JTAG mailbox FRAM bit error detection MPU segment violation	VMAIFG JMBINIFG, JMBOUTIFG CBDIFG, UBDIFG MPUSEGIIFG, MPUSEG1IFG, MPUSEG2IFG, MPUSEG3IFG (SYSSNIV) (1) (3)	(Non)maskable	0FFFCh	
User NMI External NMI Oscillator fault	NMIIFG, OFIFG (SYSUNIV) (1) (3)	(Non)maskable	0FFFAh	

Oscar Garnica. Versión 2021-22

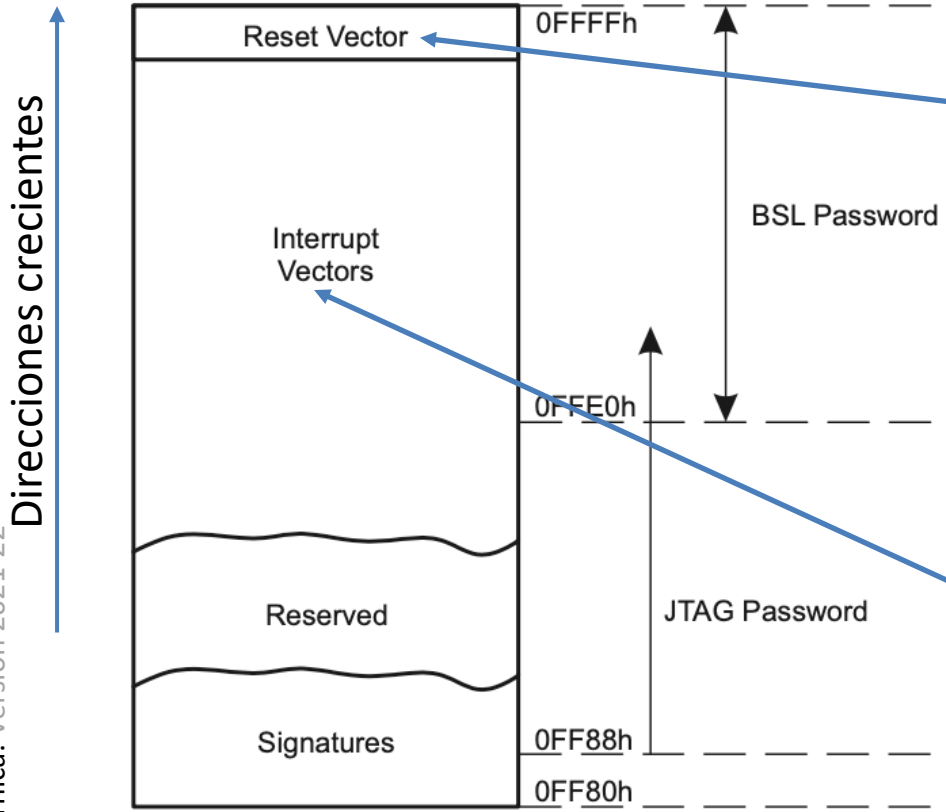
Excepciones

I/O Port P1	P1IFG.0 to P1IFG.7 (P1IV)(1)	Maskable	0FFDAh	
Timer_A TA2	TA2CCR0.CCIFG	Maskable	0FFD8h	
Timer_A TA2	TA2CCR1.CCIFG TA2CTL.TAIFG (TA2IV)(1)	Maskable	0FFD6h	
I/O Port P2	P2IFG.0 to P2IFG.7 (P2IV) (1)	Maskable	0FFD4h	

- La configuración de las excepciones se hace mediante registros
- Cada fuente de interrupción posee un conjunto de registros de configuración

Oscar Garnica. Versión 2021-22

Mapa de memoria



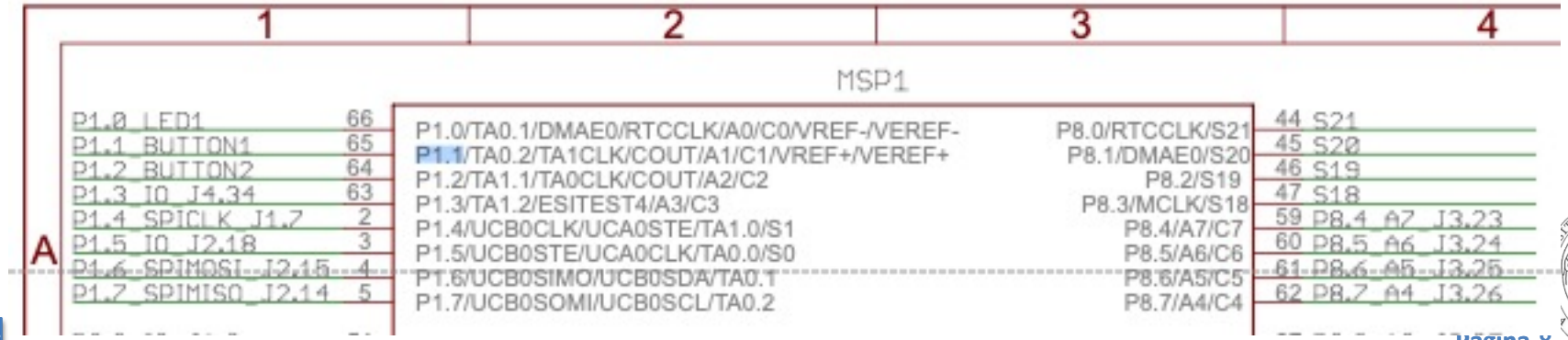
Dirección arranque después de power-up. Contiene la dirección inicial del programa de aplicación.

La tabla de vectores tendrá una entrada por vector de interrupción, que se utilizará para almacenar la dirección de la rutina que debe tratar dicha interrupción (o excepción interna)

Oscar Garnica. Versión 2021-22

Puerto P1

- Todos los pines del puerto pueden configurarse como fuentes de interrupción.
- Algunos pines ya están conectados a componentes de la placa. Por ejemplo:
  - Los botones
  - Los LEDs



Oscar Garnica. Versión 2021-22

Puerto P1. Registros de configuración

- Dirección base acceso registros P1: 01B0h

REGISTER DESCRIPTION	REGISTER	OFFSET
Port P1 input	P1IN	00h
Port P1 output	P1OUT	02h
Port P1 direction	P1DIR	04h
Port P1 resistor enable	P1REN	06h
Port P1 selection 0	P1SEL0	0Ah
Port P1 selection 1	P1SEL1	0Ch
Port P1 interrupt vector word	P1IV	0Eh
Port P1 complement selection	P1SELC	16h
Port P1 interrupt edge select	P1IES	18h
Port P1 interrupt enable	P1IE	1Ah
Port P1 interrupt flag	P1IFG	1Ch

- ¿Dónde están definidas estas direcciones?
  - msp430fr6989.h

Oscar Garnica. Versión 2021-22

Puerto P1

- Definidos en documento slau367p.pdf, pag. 388 ss
  - P1IN: registro de entrada de 8 bits
  - P1OUT: registro de salida de 8 bits
  - P1DIR: registro de dirección del puerto
  - P1REN: registro capacitación pull-up/pull-down
  - P1SEL*: selección del modulo conectado
  - P1SELC: complementa los bits in P1SEL*
  - P1IV: vector interrupciones
  - P1IES: selector tipo flanco causante INT
  - P1IE: capacita interrupciones
  - P1IFG: interrupciones pendientes

Oscar Garnica. Versión 2021-22

Puerto P1

12.4.2 PxIN Register

Port x Input Register

Figure 12-2. PxIN Register

7	6	5	4	3	2	1	0
PxIN							
r	r	r	r	r	r	r	r

Table 12-5. PxIN Register Description

Bit	Field	Type	Reset	Description
7-0	PxIN	R	Undefined	Port x input 0b = Input is low 1b = Input is high

12.4.3 PxOUT Register

Port x Output Register

Figure 12-3. PxOUT Register

7	6	5	4	3	2	1	0
PxOUT							
rw	rw	rw	rw	rw	rw	rw	rw

Table 12-6. PxOUT Register Description

Bit	Field	Type	Reset	Description
7-0	PxOUT	RW	Undefined	Port x output When I/O configured to output mode: 0b = Output is low. 1b = Output is high. When I/O configured to input mode and pullups/pulldowns enabled:

12.4.4 PxDIR Register

Port x Direction Register

Figure 12-4. PxDIR Register

7	6	5	4	3	2	1	0
PxDIR							
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

Table 12-7. P1DIR Register Description

Bit	Field	Type	Reset	Description
7-0	PxDIR	RW	0h	Port x direction 0b = Port configured as input 1b = Port configured as output

Oscar Garnica. Versión 2021-22

Manejo de los registros en C

- Variables volatile
  - Variable que puede cambiar en cualquier momento sin que el cambio sea debido a un procedimiento en el código.
- Acceso a registros del procesador
  - Usar seudo funciones. Ejemplo: __bis_sr_register
  - El compilador las sustituye por ensamblador

    __bis_SR_register(unsigned short *mask*); → BIS *mask*, SR

Oscar Garnica. Versión 2021-22

Manejo de los registros en C

- Acceso directo a registros en memoria
  - Usar directivas HWREG8, HWRE16, HWRE32
    - Definición

      #define HWREG16(x) (*((**volatile** uint16_t *)((uint16_t)x))
    - Ejemplo

      HWREG16(baseAddress + OFS_PAOUT) &= ~selectedPins
  - Usar API (gpio.h)
    - Definición: gpio.h
    - Ejemplo:

      Puerto Pines

      GPIO_setOutputLowOnPin(GPIO_PORT_P1, GPIO_PIN0 | GPIO_PIN1 | GPIO_PIN2 | GPIO_PIN3 | GPIO_PIN4 | GPIO_PIN5 | GPIO_PIN6 | GPIO_PIN7);
    - Descripción: API

Oscar Garnica. Versión 2021-22

Manejo de los registros en C

- Los identificadores de posiciones de campos de bits de los registros GPIO están definidos en gpio.h
  - GPIO_PORT_P1, GPIO_PIN0, …
  - Se pueden combinar en expresiones lógicas
- Los identificadores de posiciones de campos no asociados a ningún interfaz y las direcciones de registros están en msp430fr69898.h

Oscar Garnica. Versión 2021-22

P1. Fuentes de interrupción

- Cada uno de los 8 bits del puerto P1 puede usarse como:
  - Un pin digital de propósito general
  - Una fuente de interrupción
- La configuración del puerto se realiza escribiendo en los registros de configuración

Oscar Garnica. Versión 2021-22

RTI

- El proceso de tratamiento interrupción arranca cuando:
  - Habilitadas interrupciones en el micro (bit GIE del SR)
  - Interrupción habilitada si es enmascarable
  - Petición de interrupción
- Pasos entrada RTI
  1. Almacena PC y SR en la pila
  2. Selección INT de mayor prioridad
  3. Borrado flag petición de INT
  4. Inicialización registro de estado a 0000h. Por defecto, se deshabilita el anidamiento de interrupciones enmascarables
  5. Se obtiene dirección de comienzo de RTI
  - Latencia de 6 ciclos hasta que se ejecuta la primera instrucción de la RTI
- Pasos salida RTI
  1. Ejecución de la instrucción RETI
  2. Restauración del contexto
  - Latencia de 5 ciclos en la salida de RTI

Oscar Garnica. Versión 2021-22

RTI en C

- Pasos
  1. Habilitar globalmente interrupciones enmascarables. Dos formas, ambas basadas en seudo funciones:
     1. `_SR_register(GIE);`
     2. `__enable_interrupt();`
  2. Configurar interrupciones
     1. Definir el flanco activo (registro P*IES)
     2. Habilitar localmente la interrupción (registro P*IE)
     3. Declarar y registrar la RTI usando las siguientes dos directivas

```
 #pragma vector=[fuente de la interrupcion]
__interrupt void RTI_nombre (void) {
}
```

     4. Con el pragma, el compilador registra la RTI. Esto es, ubicar la dirección de comienzo de la RTI en la posición de memoria asignada.
     5. Con la directiva `__interrup` el compilador sabe qué prólogo y epílogo debe utilizar, los registros visibles, …

Oscar Garnica. Versión 2021-22

Ejemplo: INT asociada a botón en P1

1. El botón está asociado al pin 1 del P1
2. Configurar pin 1 del P1 como entrada de interrupción.

```
1.  P1DIR  = 0x00;     // indicar que es de entrada
2.  P1REN  = 0x02;     // pull-up/pull-down enable
3.  P1OUT  = 0x00;     // pull-up
4.  P1IES |= 0x02;    // transición de alta a baja
5.  P1IE  |= 0x02;    // habilitar interrupción local
6.  P1IFG &=~0x02;    // limpiar interrupciones pendientes
7.  _BIS_SR(GIE);     // SR habilitar interrupciones globales
```

3. Declarar RTI

```
#pragma vector=PORT1_VECTOR
__interrupt void rti_p11 (void) {
      …
      P1IFG &= ~0x02;
}
```

Definidos en msp430fr6898.h
Se puede hacer usando API.

Oscar Garnica. Versión 2021-22